

Structured Bindings can introduce a Pack

Document #: P1061R5
Date: 2023-05-17
Project: Programming Language C++
Audience: CWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Jonathan Wakely
<cxx@kayari.org>

Contents

- [1 Revision History](#)
- [2 Motivation](#)
- [3 Proposal](#)
 - [3.1 Other Languages](#)
 - [3.2 Implementation Burden](#)
 - [3.3 Implementation Experience](#)
 - [3.4 Handling non-dependent packs in the wording](#)
 - [3.5 Namespace-scope packs](#)
- [4 Wording](#)
 - [4.1 Feature-Test Macro](#)
- [5 Acknowledgements](#)
- [6 References](#)

§ 1 Revision History

R5 has minor wording changes,

R4 significantly improves the wording after review in Issaquah.

R3 removes the exclusion of namespace-scope per [EWG guidance](#).

R2 adds a section about implementation complexity, implementation experience, and wording.

R1 of this paper [[P1061R1](#)] was presented to EWG in Belfast 2019 [[P1061R1.Minutes](#)] which approved the direction as presented (12-5-2-0-1).

R0 of this paper [[P1061R0](#)] was presented to EWGI in Kona 2019 [[P1061R0.Minutes](#)], who reviewed it favorably and thought this was a good investment of our time (4-3-4-1-0). The consensus in the room was that the restriction that the introduced pack need not be the trailing identifier.

§ 2 Motivation

Function parameter packs and tuples are conceptually very similar. Both are heterogeneous sequences of objects. Some problems are easier to solve with a parameter pack, some are easier to solve with a tuple. Today, it's trivial

to convert a pack to a tuple, but it's somewhat more involved to convert a tuple to a pack. You have to go through `std::apply()` [N3915]:

```
std::tuple<A, B, C> tup = ...;
std::apply([&](auto&&... elems){
    // now I have a pack
}, tup);
```

This is great for cases where we just need to call a [non-overloaded] function or function object, but rapidly becomes much more awkward as we dial up the complexity. Not to mention if I want to return from the outer scope based on what these elements have to be.

How do we compute the dot product of two tuples? It's a choose your own adventure of awkward choices:

Nested <code>apply()</code>	Using <code>index_sequence</code>
<pre>template <class P, class Q> auto dot_product(P p, Q q) { return std::apply([&](auto... p_elems) { return std::apply([&](auto... q_elems) { return (... + (p_elems * q_elems)); }, q); }, p); }</pre>	<pre>template <size_t... Is, class P, class Q> auto dot_product(std::index_sequence<Is...>, P p, Q, q) { return (... + (std::get<Is>(p) * std::get<Is>(q))); } template <class P, class Q> auto dot_product(P p, Q q) { return dot_product(std::make_index_sequence<std::tuple_size<P>::value> {}), p, q); }</pre>

Regardless of which option you dislike the least, both are limited to only `std::tuple`s. We don't have the ability to do this at all for any of the other kinds of types that can be used in a structured binding declaration [P0144R2] - because we need to explicit list the correct number of identifiers, and we might not know how many there are.

§ 3 Proposal

We propose to extend the structured bindings syntax to allow the user to introduce a pack as (at most) one of the identifiers:

```
std::tuple<X, Y, Z> f();

auto [x,y,z] = f();           // OK today
auto [...xs] = f();           // proposed: xs is a pack of length three containing an X, Y, and
    a Z
auto [x, ...rest] = f();       // proposed: x is an X, rest is a pack of length two (Y and Z)
auto [x,y,z, ...rest] = f();   // proposed: rest is an empty pack
auto [x, ...rest, z] = f();    // proposed: x is an X, rest is a pack of length one
                                // consisting of the Y, z is a Z
auto [...a, ...b] = f();       // ill-formed: multiple packs
```

If we additionally add the structured binding customization machinery to `std::integer_sequence`, this could greatly simplify generic code:

Today	Proposed
Implementing <code>std::apply()</code>	
<pre> namespace detail { template <class F, class Tuple, std::size_t... I> constexpr decltype(auto) apply_impl(F &&f, Tuple &&t, std::index_sequence<I...>) { return std::invoke(std::forward<F>(f), std::get<I>(std::forward<Tuple>(t))...); } } template <class F, class Tuple> constexpr decltype(auto) apply(F &&f, Tuple &&t) { return detail::apply_impl(std::forward<F>(f), std::forward<Tuple>(t), std::make_index_sequence<std::tuple_size_v< std::decay_t<Tuple>>>{}); } </pre>	<pre> template <class F, class Tuple> constexpr decltype(auto) apply(F &&f, Tuple &&t) { auto&& [...elems] = t; return std::invoke(std::forward<F> (f), forward_like<Tuple, decltype(elems)> (elems)...); } </pre>
<code>dot_product()</code> , nested	
<pre> template <class P, class Q> auto dot_product(P p, Q q) { return std::apply([&](auto... p_elems){ return std::apply([&](auto... q_elems){ return (... + (p_elems * q_elems)); }, q) }, p); } </pre>	<pre> template <class P, class Q> auto dot_product(P p, Q q) { // no indirection! auto&& [...p_elems] = p; auto&& [...q_elems] = q; return (... + (p_elems * q_elems)); } </pre>
<code>dot_product()</code> , with <code>index_sequence</code>	
<pre> template <size_t... Is, class P, class Q> auto dot_product(std::index_sequence<Is...>, P p, Q, q) { return (... + (std::get<Is>(p) * std::get<Is> (q))); } template <class P, class Q> auto dot_product(P p, Q q) { return dot_product(std::make_index_sequence<std::tuple_size_v<P>>{} {}, p, q); } </pre>	<pre> template <class P, class Q> auto dot_product(P p, Q q) { // no helper function // necessary! auto [...Is] = std::make_index_sequence< std::tuple_size_v<P>>{}; return (... + (std::get<Is>(p) * std::get<Is>(q))); } </pre>

Not only are these implementations more concise, but they are also more functional. I can just as easily use `apply()` with user-defined types as I can with `std::tuple`:

```

struct Point {
    int x, y, z;
};

```

```
Point getPoint();
double calc(int, int, int);

double result = std::apply(calc, getPoint()); // ill-formed today, ok with proposed
implementation
```

§ 3.1 Other Languages

Python 2 had always allowed for a syntax similar to C++17 structured bindings, where you have to provide all the identifiers:

```
>>> a, b, c, d, e = range(5) # ok
>>> a, *b = range(3)
File "<stdin>", line 1
  a, *b = range(3)
      ^
SyntaxError: invalid syntax
```

But you could not do any more than that. Python 3 went one step further by way of PEP-3132 [[PEP.3132](#)]. That proposal allowed for a single starred identifier to be used, which would bind to all the elements as necessary:

```
>>> a, *b, c = range(5)
>>> a
0
>>> c
4
>>> b
[1, 2, 3]
```

The Python 3 behavior is synonymous with what is being proposed here. Notably, from that PEP:

Possible changes discussed were:

- Only allow a starred expression as the last item in the exprlist. This would simplify the unpacking code a bit and allow for the starred expression to be assigned an iterator. This behavior was rejected because it would be too surprising.

R0 of this proposal only allowed a pack to be introduced as the last item, which was changed in R1.

§ 3.2 Implementation Burden

Unfortunately, this proposal has some implementation complexity. The issue is not so much this aspect:

```
template <typename Tuple>
auto sum_template(Tuple tuple) {
    auto [...elems] = tuple;
    return (... + elems);
}
```

This part is more or less straightforward - we have a dependent type and we introduce a pack from it, but we're already in a template context where dealing with packs is just a normal thing.

The problem is this aspect:

```
auto sum_non_template(SomeConcreteType tuple) {
    auto [...elems] = tuple;
```

```

    return (... + elems);
}

```

We have not yet in the history of C++ had this notion of packs outside of dependent contexts. This is completely novel, and imposes a burden on implementations to have to track packs outside of templates where they previously had not.

However, in our estimation, this functionality is going to come to C++ in one form or other fairly soon. Reflection, in the latest form of [P1240R2], has many examples of introducing packs in non-template contexts as well - through the notion of a *reflection range*. That paper introduces several reifiers that can manipulate a newly-introduced pack, such as:

```

std::meta::info t_args[] = { ^int, ^42 };
template<typename T, T> struct X {};
X<...[:t_args:]...> x; // Same as "X<int, 42> x;".
template<typename, typename> struct Y {};
Y<...[:t_args:]...> y; // Error: same as "Y<int, 42> y;".

```

As with the structured bindings example in this paper - we have a non-dependent object outside of a template that we're using to introduce a pack.

Furthermore, unlike some of the reflection examples, and some of the more generic pack facilities proposed in [P1858R2], this paper offers a nice benefit: all packs must still be declared before use. Even in the `sum_non_template` example which, as the name suggests, is not a template in any way, the pack `elems` needs an initial declaration. So any machinery that implementations need to track packs doesn't need to be enabled everywhere - only when a pack declaration has been seen.

§ 3.3 Implementation Experience

Jason Rice has implemented this in a [clang](#). As far as we've been able to ascertain, it works great.

It is also [available on Compiler Explorer](#).

§ 3.4 Handling non-dependent packs in the wording

The strategy the wording takes to handle `sum_non_template` above is to designate `elems` as a *non-dependent pack* and to state that non-dependent packs are instantiated immediately. In that example, `elems` is obviously non-dependent (nothing anywhere is dependent), so we just instantiate the *fold-expression* immediately.

But defining “non-dependent pack” is non-trivial. Examples from Richard Smith:

```

template<int> struct X { using type = int; };
template<typename T> void f() {
    struct Y { int a, b; };
    auto [...v] = Y();
    X<sizeof...(v)>::type x; // need typename or no?
}

```

Is `X<sizeof...(v)>` a dependent type or is the pack `v` expanded eagerly because we already know its size?

One approach could be to say that packs are instantiated immediately only if they appear outside of *any* template. But then:

```

struct Z { int a, b; };
template <typename T> void g() {
    auto [...v] = Z();
    X<sizeof...(v)>::type x; // need typename or no?
}

```

Here, despite being in a template, nothing is actually dependent.

I think the right line to draw here is to follow [\[CWG2074\]](#) - which asks about this example:

```

template<typename T>
void f() {
    struct X {
        typedef int type;
#ifdef DEPENDENT
        T x;
#endif
    };
    X::type y;    // #1
}

void g() { f<int>(); }

```

there is implementation variance in the treatment of `#1`, but whether or not `DEPENDENT` is defined appears to make no difference.

[...]

Perhaps the right answer is that the types should be dependent but a member of the current instantiation, permitting name lookup without `typename`.

I think the best rule would be to follow the suggested answer in this core issue: a structured binding pack is dependent if: the type of its initializer (the `E` in 9.6 [\[dcl.struct.bind\]](#)) is dependent and it is not a member of the current instantiation. This would make neither of the `...v` packs dependent, which seems conceptually correct.

§ 3.5 Namespace-scope packs

In addition to non-dependent packs, this paper also seems like it would offer the ability to declare a pack at *namespace* scope:

```

struct Point { int x, y; };
auto [... parts] = Point{1, 2};

```

Structured bindings in namespace scope are a little odd to begin with, since they currently cannot be declared `inline`. A structured binding pack at namespace scope adds that much more complexity. However, rejecting it would be a very odd kind of restriction that would be surprising. If users want to do this, they should be able to. EWG also explicitly polled this question in a telecon, with a strong preference for allowing such namespace-scope declarations.

§ 4 Wording

Add a new grammar option for *simple-declaration* to 9.1 [\[dcl.pre\]](#):

sb-pack-identifier:

... identifier

sb-identifier-list:

identifier

sb-pack-identifier

sb-identifier-list , identifier

sb-identifier-list , sb-pack-identifier

simple-declaration:

*decl-specifier-seq init-declarator-list*_{opt} ;

attribute-specifier-seq decl-specifier-seq init-declarator-list ;

*attribute-specifier-seq*_{opt} *decl-specifier-seq ref-qualifier*_{opt} [*identifier-list sb-identifier-list*] *initializer* ;

Change 9.1 [dcl.pre] paragraph 8:

- 8 A *simple-declaration* with an *identifier-list sb-identifier-list* is called a structured binding declaration ([dcl.struct.bind]). The *decl-specifier-seq* shall contain only the *type-specifier* *auto* and *cv-qualifiers*. The *initializer* shall be of the form “= *assignment-expression*”, of the form “{ *assignment-expression* }”, or of the form “(*assignment-expression*)”, where the *assignment-expression* is of array or non-union class type.

Extend 9.3.4.6 [dcl.fct]/5:

- 5 The type of a function is determined using the following rules. The type of each parameter (including function parameter packs, *after immediately expanding structured binding packs* ([temp.variadic])) is determined from its own *parameter-declaration* ([dcl.decl]). After determining the type of each parameter, any parameter of type “array of T” or of function type T is adjusted to be “pointer to T”. After producing the list of parameter types, any top-level *cv-qualifiers* modifying a parameter type are deleted when forming the function type. The resulting list of transformed parameter types and the presence or absence of the ellipsis or a function parameter pack is the function’s parameter-type-list.

[Note 3: This transformation does not affect the types of the parameters. For example, *int(*) (const int p, decltype(p)*)* and *int(*) (int, const int*)* are identical types. — end note] [Example 2:

```
void f(char*);           // #1
void f(char[]) {}        // defines #1
void f(const char*) {}   // OK, another overload
void f(char *const) {}   // error: redefines #1

void g(char(*)[2]);       // #2
void g(char[3][2]) {}     // defines #2
void g(char[3][3]) {}     // OK, another overload

void h(int x(const int)); // #3
void h(int (*)(int)) {}   // defines #3

+ struct C {
+     int i;
+     long l;
+ };
```

```
+ auto [...v] = C{ 1, 0 };
+ C k(int, long);           // #4
+ C k(decltype(v)... p) {   // defines #4
+     return C{p...};
+ }
```

— end example]

Change 9.6 [dcl.struct.bind] paragraph 1:

- 1 A structured binding declaration introduces the *identifiers* $v_0, v_1, v_2, \dots, v_{N-1}$ of the *identifier-list* *sb-identifier-list* as names ([basic.scope.declarative]) *of structured bindings*. A structured binding is either an *identifier* of the *sb-identifier-list* or an element of the pack introduced by an *sb-pack-identifier*. The declaration shall contain at most one *sb-pack-identifier*. Let *cv* denote the *cv-qualifiers* in the *decl-specifier-seq*.

Introduce new paragraphs after 9.6 [dcl.struct.bind] paragraph 1, introducing the terms “structured binding size” and SB_i :

- 1+1 The *structured binding size* of a type E is the required number of names that need to be introduced by the structured binding declaration, as defined below. If there is no structured binding pack, then the number of elements in the *sb-identifier-list* shall be equal to the structured binding size. Otherwise, the number of elements of the structured binding pack is the structured binding size less the number of non-pack elements in the *sb-identifier-list* and the number of non-pack elements shall be no more than the structured binding size.

- 1+2 Let SB_i denote the i^{th} structured binding in the structured binding declaration after expanding the structured binding pack, if any. [*Note*: If there is no structured binding pack, then SB_i denotes v_i . - *end note*] [*Example*:

```
struct C { int x, y, z; };

auto [a, b, c] = C(); //  $SB_0$  is a,  $SB_1$  is b, and  $SB_2$  is c
auto [d, ...e] = C(); //  $SB_0$  is d, the pack e ( $v_1$ ) contains two identifiers:  $SB_1$  and  $SB_2$ 
auto [...f, g] = C(); // the pack f ( $v_0$ ) contains two identifiers:  $SB_0$  and  $SB_1$ , and  $SB_2$  is g
auto [h, i, j, k, ...l] = C(); // error: structured binding size is too small

— end example ]
```

Change 9.6 [dcl.struct.bind] paragraph 3 to define a structured binding size and extend the example:

- 3 If E is an array type with element type T , ~~the number of elements in the identifier-list shall be the~~ *structured binding size of E* is equal to the number of elements of E . Each v_i SB_i is the name of an lvalue that refers to the element i of the array and whose type is T ; the referenced type is T . [Note: The top-level *cv-qualifiers* of T are *cv*. — *end note*] [Example:

```
auto f() -> int(&)[2];
auto [ x, y ] = f();           // x and y refer to elements in a copy of the array
    return value
auto& [ xr, yr ] = f();        // xr and yr refer to elements in the array
    referred to by f's return value
```



```
+ auto g() -> int(&)[4];
+ auto [a, ...b, c] = g();    // a is the first element of the array, b is a pack
    containing the second and
+                               // third elements, and c is the fourth element
+ auto& [...e] = g();        // e is a pack referring to the four elements of
    the array
```

— end example]

Change 9.6 [dcl.struct.bind] paragraph 4 to define a structured binding size:

- 4 Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete type, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the ~~number of elements in the identifier-list shall be~~ **structured binding size of E** is equal to the value of that expression. [...] Each ~~v_i~~ **SB_i** is the name of an lvalue of type T_i that refers to the object bound to r_i ; the referenced type is T_i .

Change 9.6 [dcl.struct.bind] paragraph 5 to define a structured binding size:

- 5 Otherwise, all of E's non-static data members shall be direct members of E or of the same base class of E, well-formed when named as `e.name` in the context of the structured binding, E shall not have an anonymous union member, and the ~~number of elements in the identifier-list shall be~~ **structured binding size of E** is equal to the number of non-static data members of E. Designating the non-static data members of E as $m_0, m_1, m_2, \dots$ (in declaration order), each ~~v_i~~ **SB_i** is the name of an lvalue that refers to the member m_i of E and whose type is `cv  $T_i$` , where T_i is the declared type of that member; the referenced type is `cv  $T_i$` . The lvalue is a bit-field if that member is a bit-field.

Add a new clause to 13.7.4 [temp.variadic], after paragraph 3:

- 3+ A **structured binding pack** is an identifier that introduces zero or more **structured bindings** ([dcl.struct.bind]). [Example

```
auto foo() -> int(&)[2];
auto [...a] = foo();    // a is a structured binding pack containing 2
    elements
auto [b, c, ...d] = foo(); // d is a structured binding pack containing 0
    elements
```

— end example]

In 13.7.4 [temp.variadic], change paragraph 4:

- 4 A **pack** is a template parameter pack, a function parameter pack, ~~or~~ an *init-capture* pack, **or a structured binding pack**. The number of elements of a template parameter pack or a function parameter pack is the number of arguments provided for the parameter pack. The number of elements of an *init-capture* pack is the number of elements in the pack expansion of its *initializer*.

In 13.7.4 [temp.variadic], paragraph 5 (describing pack expansions) remains unchanged.

In 13.7.4 [temp.variadic], add a bullet to paragraph 8:

- 8 Such an element, in the context of the instantiation, is interpreted as follows:

- (8.1) — if the pack is a template parameter pack, the element is a template parameter ([temp.param]) of the corresponding kind (type or non-type) designating the i^{th} corresponding type or value template argument;
- (8.2) — if the pack is a function parameter pack, the element is an *id-expression* designating the i^{th} function parameter that resulted from instantiation of the function parameter pack declaration; **otherwise**
- (8.3) — if the pack is an *init-capture* pack, the element is an *id-expression* designating the variable introduced by the i^{th} *init-capture* that resulted from instantiation of the *init-capture* pack; **otherwise**
- (8.4) — **if the pack is a structured binding pack, the element is an *id-expression* designating the i^{th} structured binding that resulted from the structured binding declaration.**

Add a new paragraph after 13.7.4 [\[temp.variadic\]](#)

- 12 **If all of the packs expanded by a pack expansion are structured binding packs which were initialized with non-dependent *brace-or-equal-initializers*, the pack expansion is expanded immediately.**

[*Example:*

```
struct C { };

void g(...); // #1

template <typename T>
void f() {
    C arr[1];
    auto [...e] = arr;
    g(e...); // calls #1
}

void g(C); // #2

int main() {
    f<int>();
}

— end example ]
```

Add a bullet to 13.8.3.3 [\[temp.dep.expr\]](#)/3:

- 3 **An *id-expression* is type-dependent if it is a *template-id* that is not a concept-id and is dependent; or if its terminal name is**
 - (3.1) — associated by name lookup with one or more declarations declared with a dependent type,
 - (3.2) — associated by name lookup with a non-type template-parameter declared with a type that contains a placeholder type,
 - (3.3) — associated by name lookup with a variable declared with a type that contains a placeholder type ([dcl.spec.auto]) where the initializer is type-dependent,
 - (3.4) — associated by name lookup with one or more declarations of member functions of a class that is the current instantiation declared with a return type that contains a placeholder type,

- (3.5) — associated by name lookup with a structured binding declaration ([`dcl.struct.bind`]) whose brace-or-equal-initializer is type-dependent,
- (3.5b) — associated by name lookup with a pack, unless that pack is a non-type template parameter pack whose types are non-dependent,
- (3.6) — associated by name lookup with an entity captured by copy ([`expr.prim.lambda.capture`]) in a lambda-expression that has an explicit object parameter whose type is dependent ([`dcl.fct`]),
- (3.7) — the identifier **func** ([`dcl.fct.def.general`]), where any enclosing function is a template, a member of a class template, or a generic lambda,
- (3.8) — a conversion-function-id that specifies a dependent type, or
- (3.9) — dependent

Add a carve-out for in 13.8.3.4 [[temp.dep.constexpr](#)]/4:

- 4 Expressions of the following form are value-dependent:

```
sizeof ... ( identifier )
fold-expression
```

unless the *identifier* is a structured binding pack initialized with a non-dependent *brace-or-equal-initializer*, or all of the packs expanded in the *fold-expression* are structured binding packs initialized with a non-dependent *brace-or-equal-initializer*.

§ 4.1 Feature-Test Macro

Bump `__cpp_structured_bindings` in 15.11 [[cpp.predefined](#)]:

```
- __cpp_structured_bindings 201606L
+ __cpp_structured_bindings 2023XXL
```

§ 5 Acknowledgements

Thanks to Michael Park and Tomasz Kamiński for their helpful feedback. Thanks to Richard Smith for help with the wording. Thanks especially to Jason Rice for the implementation.

§ 6 References

[CWG2074] Richard Smith. 2015-01-20. Type-dependence of local class of function template.

<https://wg21.link/cwg2074>

[N3915] Peter Sommerlad. 2014-02-14. `apply()` call a function with arguments from a tuple (V3).

<https://wg21.link/n3915>

[P0144R2] Herb Sutter. 2016-03-16. Structured Bindings.

<https://wg21.link/p0144r2>

[P1061R0] Barry Revzin, Jonathan Wakely. 2018-05-01. Structured Bindings can introduce a Pack.

<https://wg21.link/p1061r0>

[P1061R0.Minutes] EWGI. 2019. Kona 2019 EWGI: P1061R0.

<http://wiki.edg.com/bin/view/Wg21kona2019/P1061>

[P1061R1] Barry Revzin, Jonathan Wakely. 2019-10-07. Structured Bindings can introduce a Pack.

<https://wg21.link/p1061r1>

[P1061R1.Minutes] EWG. 2019. Belfast 2020 EWG: P1061R1.

<https://wiki.edg.com/bin/view/Wg21belfast/P1061-EWG>

[P1240R2] Daveed Vandevoorde, Wyatt Childers, Andrew Sutton, Faisal Vali. 2022-01-14. Scalable Reflection.

<https://wg21.link/p1240r2>

[P1858R2] Barry Revzin. 2020-03-01. Generalized pack declaration and usage.

<https://wg21.link/p1858r2>

[PEP.3132] Georg Brandl. 2007. PEP 3132 – Extended Iterable Unpacking.

<https://www.python.org/dev/peps/pep-3132/>